

FPGA BASED REAL-TIME DETECTION PIPELINE FOR CHALLENGING ENVIRONMENTS

By Aditya Roy Chowdhury

Place of Work: The International Centre for Theoretical Physics, Trieste, Italy

Submitted to the Computer Science & Engineering Department, Thapar Institute of Engineering & Technology

June 2024

In Partial Fulfilment of the Requirements for the Degree of Bachelor of Engineering in Computer Engineering

Abstract

With the growth in demand for Artificial Intelligence and Machine Learning, Object Detection has become a field of active research. Its importance in advanced driver-assistance systems (ADAS) can not be neglected. A useful ADAS using object detection needs to be robust to weather conditions and should execute inference efficiently. This is where the research described in this report aimed to contribute. In this research, the SpineNet family of backbone architecture was considered for deployment on the edge. An effort was made to import the SpineNet 49S architecture on the Field Programmable Gated Arrays(FPGA) board using Vivado HLS. However, SpineNet proved to be a huge model for an FPGA. Since FPGA devices present resource limitations, the study also delved into studying the possible compression that can be imposed on the backbone before migrating it to an FPGA. The research uses the Cityscape dataset along with a version of it that has synthetic haze added to it. The dataset is used to create baselines and execute the distillation of the backbone into a student model. The work culminated with a custom SpineNet implementation for easier experimentation and the first iteration of a smaller student model inspired by SpineNet 49S, along with other contributions such as parsers and format converters.



Author Aditya Roy Chowdhury



Certified by Dr. Maria Liz Crespo
(Industrial Mentor)



Certified by Dr. Rajkumar Tekchandani
(Faculty Mentor)

science, technology and innovation unit
e-mail: sti@ictp.it

14 May 2024

TO WHOM IT MAY CONCERN

This is to certify that

Mr Aditya ROY CHOWDHURY

visited ICTP from 6 February to 26 April 2024 and carried out an internship on FPGA based Real-Time Detection Pipeline for Challenging Environments under the supervision of Dr. Maria Liz Crespo at the Multidisciplinary Laboratory (MLab) within the Science, Technology and Innovation Unit (STI) of the ICTP.



Dr. Maria Liz Crespo
Associate Research Scientist
Coordinator,
Multidisciplinary Lab
Science, Technology and Innovation Unit

The ICTP MLab is a Multidisciplinary Laboratory for research, training, and development of advanced scientific instrumentation. Discover more at: <https://www.ictp.it/sti>

Strada Costiera 11, I-34151 Trieste, Italy • Tel. +39 040 2240111 • pio@ictp.it • www.ictp.it
ICTP is governed by the IAEA, Italy and UNESCO, and is a UNESCO Category 1 Institute

TABLE OF CONTENT

1. COMPANY PROFILE.....	5
2. INTRODUCTION.....	6
3. BACKGROUND.....	6
3.1.COMPLETE PIPELINE.....	6
3.2.BACKBONE.....	7
3.3.FEATURE PYRAMID NETWORK.....	9
3.4.RETINANET.....	10
3.5.DATASET.....	11
3.6.KNOWLEDGE DISTILLATION FOR OD.....	13
3.7.QUANTIZATION.....	13
4. PROBLEM STATEMENT.....	14
4.1.MOTIVATION.....	14
4.2.CHALLENGES.....	14
5. OBJECTIVES.....	15
6. METHODOLOGY.....	15
7. OBSERVATIONS AND FINDINGS.....	18
7.1.FOR OBJECTIVE 1.....	18
7.2.C SYNTHESIS.....	19
7.3.SPINENET49S IMPLEMENTATION.....	20
7.4.STUDENT.....	20
7.5.QUANTIZATION.....	21
8. PROGRESS.....	22
9. LIMITATIONS.....	23
10.CONCLUSIONS	24
11.FUTURE WORK.....	25
12.REFERENCES.....	26
13. ANNEXURE A: PEER REVIEW FORM.....	27

1. COMPANY PROFILE

The International Centre for Theoretical Physics (ICTP) is a globally renowned research institution that fosters advanced studies and research in physical and mathematical sciences. Established in 1964 by the late Nobel Laureate Abdus Salam, ICTP operates under a tripartite agreement between the Italian Government, the International Atomic Energy Agency (IAEA), and the United Nations Educational, Scientific and Cultural Organization (UNESCO). The center's primary goal is to bridge the scientific gap between developed and developing countries by providing scientists from around the world with access to high-level research facilities and opportunities for collaboration.

Alumni of the ICTP program work as professors at prestigious institutions, academic department chairs, research center directors, and science and technology ministers in developing countries. Numerous of them have received recognition for their contributions to science and science policy both domestically and globally. The effects of ICTP are felt globally.

The Science, Technology, and Innovation (STI) Unit is a specialized division within ICTP that focuses on leveraging science and technology to address global challenges and promote sustainable development.

It also aims to enhance the capacity of developing countries in science, technology, and innovation. It promotes the use of technology to solve socio-economic and environmental challenges, supports the development of innovative solutions and technological entrepreneurship, and facilitates technology transfer.

The unit engages in several key activities to achieve its mission. It offers specialized training programs and workshops to build skills in areas such as computer science, engineering, biotechnology, and renewable energy. These programs are designed to equip participants with the knowledge and skills needed to drive technological innovation in their respective countries.

In collaboration with the Thapar Institute of Engineering and Technology, the STI unit undertook the “FPGA BASED REAL-TIME DETECTION PIPELINE FOR CHALLENGING ENVIRONMENTS” research project by contributing its resources and world-class researchers.

2. INTRODUCTION

In an era where autonomous vehicles and advanced driver assistance systems are rapidly evolving, reliably and quickly detecting vehicles in adverse weather conditions remains a critical challenge. From dense fog to heavy rain, these environmental factors can significantly impair the performance of conventional vehicle detection systems, posing serious safety concerns on our roads.

This project, undertaken by The Multidisciplinary Lab of The International Centre for Theoretical Physics, aims to study and aid the mentioned necessity. The project aims to migrate the backbone onto an FPGA for faster inferences. Implementing such a network on FPGA will be the first of its kind. The primary task of the conducted research is to work towards a backbone inspired by the SpineNet architecture that is suitable for FPGA deployment (smaller size and similar performance). Model compression methods, such as Network Pruning, Knowledge Distillation, etc., are considered for size reduction [5].

3. BACKGROUND

This section thoroughly discusses various elements related to the project. It begins with an overview of the network components, detailing their roles and functionalities within the system. Following this, an in-depth description of the dataset used is provided, including its sources, structure, and relevance to the project objectives. The methodologies and techniques employed throughout the project are also elaborated upon, highlighting the processes and tools that facilitated the research and development. Furthermore, this section delves into the theoretical framework that underpins the entire project, offering insights into the principles and concepts that guided the approach and execution.

3.1 COMPLETE PIPELINE

The proposed solution (Fig 1) is a pipeline that inputs the hazy/foggy scene and gives the detections' bounding boxes and confidence scores. The project focuses on the backbone deployment, leaving the neck and head as possible future work.

The backbone network, responsible for extracting the feature pyramid, will be deployed on an FPGA for real-time inference capabilities. Subsequently, RetinaNet is used as the detector, which uses the feature pyramid to output the bounding boxes and classification results.

SpineNet performs better than Resnet on clear data. However, the choice of SpineNet as the backbone is primarily based on its superior performance on hazy datasets despite its training on clear datasets.

Backbone Model	FLOP's	#Params	AP
R50-FPN	96.8B	34.0M	40.4
SpineNet 49	85.4B	28.5M	42.8
SpineNet 49S	33.8	12.0M	39.5

Table 1: Information on models

Additionally, the SpineNet49[1] model performs better than Resnet50 and requires fewer FLOPS. However, due to memory restrictions of FPGA boards, a scaled-down variant of the SpineNet49 backbone, SpineNet49S, has been considered for this study. SpineNet 49S provides similar precision for approximately one-third of FLOPs compared to Resnet50, as seen in Table 1.

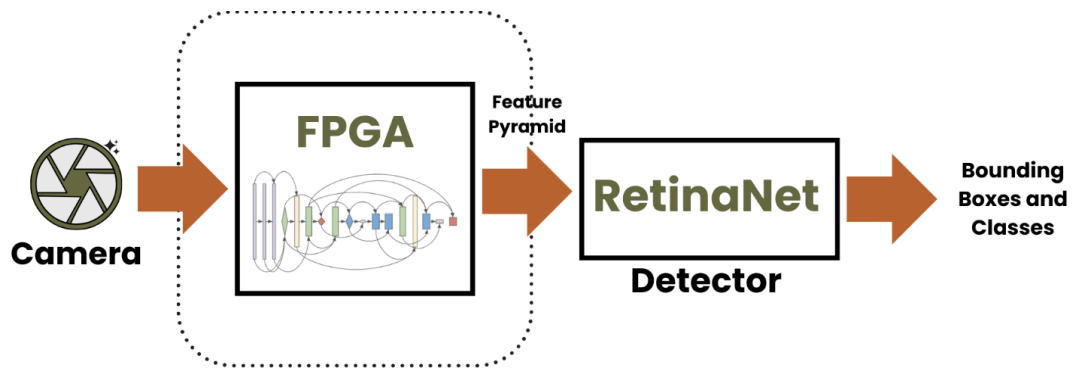


Fig 1 DETECTION PIPELINE
The dotted square denotes the project work.

3.2 BACKBONE

SpineNet, depicted in Fig 2, is a family of scale-permuted backbone architectures with better performance for localization tasks and significantly fewer floating point operations per second (FLOPS) than Resnet50 [2] (Resnet uses 96.8B FLOPS and gives 42.3 AP. SpineNet49S uses 33.8B FLOPS and gives 41.5 AP) [1].

2. Cross-scale connections[1]:

These connections directly link features across different resolutions, further facilitating information flow and boosting performance.

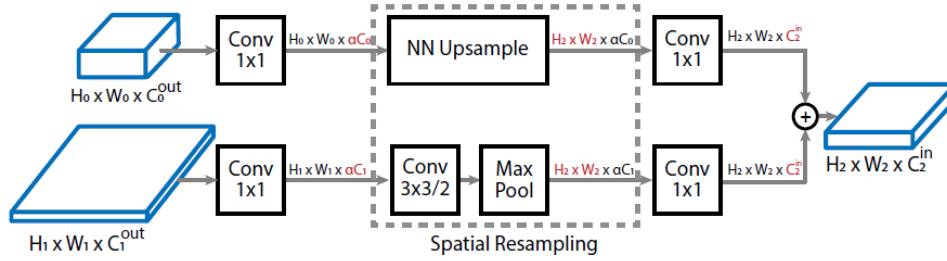


Fig 4

3. Neural Architecture Search (NAS)[1]:

The network architecture is learned automatically through NAS, leading to an efficient and effective design. The search space includes scale permutations, which rearrange the order of intermediate and output building blocks to optimize their processing sequence. Secondly, it involves cross-scale connections, allowing each block to connect to any lower-ordered block or the stem network, facilitating diverse feature fusion. Lastly, the search space includes block adjustments, enabling blocks to modify their scale level and type for fine-tuning their properties.

3.3 FPN(FEATURE PYRAMID NETWORK)

Feature Pyramid Networks (FPN) are a type of neural network architecture commonly used in computer vision tasks, particularly in object detection and instance segmentation. FPN addresses the challenge of detecting objects at multiple scales within an image. It achieves this by constructing a feature pyramid with multiple levels of semantic information, where each level corresponds to a different scale (Fig 5). This allows the network to detect objects of various sizes more effectively. FPN has been widely adopted in state-of-the-art object detection systems, contributing to improved accuracy and robustness in detecting objects in complex scenes.

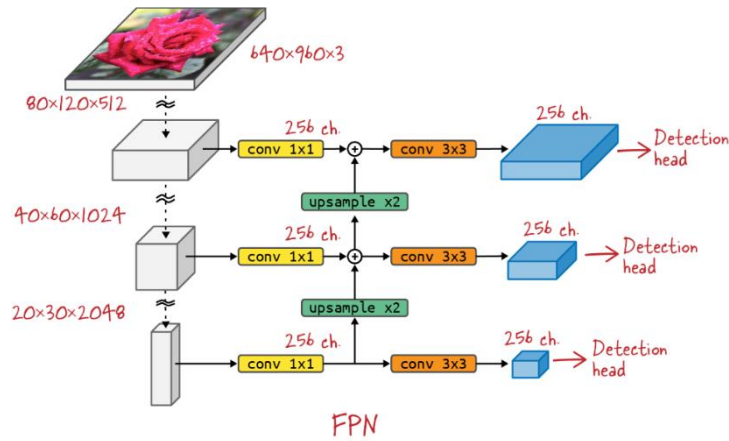


Fig 5. FPN

3.4 RETINANET

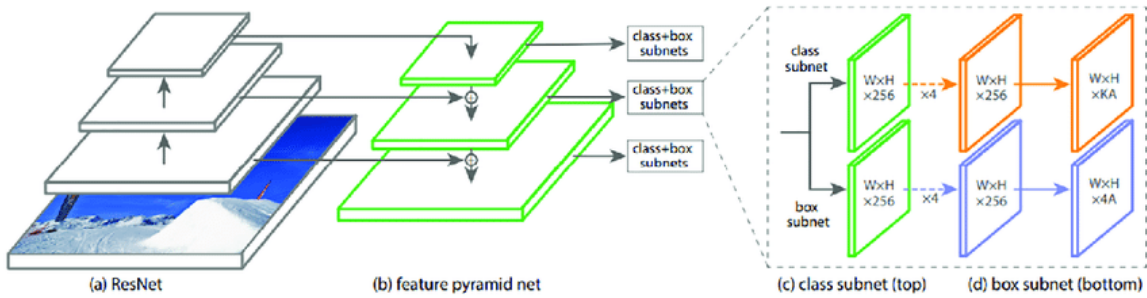


Fig 6 RetinaNet

RetinaNet is a popular object detection head renowned for its ability to accurately detect objects at various scales in images, particularly in scenarios where objects have different sizes. It addresses the limitations of previous object detection models by introducing a novel architecture that combines a feature pyramid network (FPN) with a simple and efficient detection subnetwork.

The key components and characteristics of RetinaNet are as follows:

- Feature Pyramid Network (FPN):

RetinaNet utilizes FPN to build a feature pyramid with multiple levels of semantic information. This allows the model to capture objects of different scales effectively, ensuring robust detection across the entire image.

- Anchor Boxes:

Similar to other object detection models, RetinaNet employs anchor boxes to generate candidate object bounding boxes at different locations and scales across the image. These anchor boxes serve as reference frames for object detection.

- Classification and Regression Subnetworks:

RetinaNet introduces two parallel subnetworks for object detection. The classification subnetwork predicts the probability of object presence within each anchor box, while the regression subnetwork predicts the bounding box coordinates for each anchor box.

- Focal Loss:

One of the key innovations of RetinaNet is the use of focal loss, which addresses the class imbalance problem inherent in object detection datasets. Focal loss assigns higher weights to hard-to-classify examples, focusing the model's attention on challenging cases and improving overall performance.

- Efficient Training and Inference:

Despite its high performance, RetinaNet maintains efficiency in both training and inference. The model achieves state-of-the-art results with relatively fewer computational resources compared to more complex architectures.

3.5 DATASET

The Cityscapes dataset[3] is a widely used benchmark dataset in computer vision. It comprises high-resolution images, as shown in Fig 7, captured in urban street scenes from various cities, annotated with pixel-level semantic labels such as road, sidewalk, building, person, car, etc. The dataset provides rich and diverse data for tasks such as semantic segmentation, instance segmentation, and object detection.

Foggy Cityscapes[4] is an extension of the Cityscapes dataset, where synthetic fog is added to the original images to simulate adverse weather conditions. This augmentation aims to improve the robustness of models trained on the dataset by exposing them to challenging environmental conditions encountered in real-world scenarios, such as foggy or misty weather. Foggy

Cityscapes thus provides a more comprehensive evaluation benchmark for algorithms designed to operate in urban environments under varying weather conditions.



Fig 7. Original data



Fig 8. Foggy data with three intensities of haze

3.6 KNOWLEDGE DISTILLATION FOR OBJECT DETECTION

A well-known training technique in deep neural networks is called Knowledge Distillation (KD), which transfers knowledge from a big teacher model to a smaller student. KD has shown to be a successful method for greatly enhancing students' performance on a variety of tasks.

However, KD has had limitations with object detection tasks(OD) as OD has a more complex output space[5][6]. Typically, KD is approached by utilizing the network's final outputs.

Applying KD at the intermediate feature level is required due to the OD model's complex output space, while output-based KD alone has been shown to be ineffective[6][7]. Additionally, instead of comparing the intermediate feature on a pixel basis using the lp-norm, l_{ssim} [6] is used to compare patches of feature maps. This l_{ssim} is the loss function, which is minimized during the distillation process.

$$l = \frac{2\mu_S\mu_T + C_1}{\mu_S^2 + \mu_T^2 + C_1} \quad c = \frac{2\sigma_S\sigma_T + C_2}{\sigma_S^2 + \sigma_T^2 + C_2} \quad s = \frac{\sigma_{ST} + C_3}{\sigma_S\sigma_T + C_3}$$

$$\ell_{SSIM} : \mathcal{L}_\epsilon = (1 - SSIM)/2 = (1 - (l^\alpha \cdot c^\beta \cdot s^\gamma))/2$$

$C_1 = (K_1L)^2$, $C_2 = (K_2L)^2$, $C_3 = C_2/2$, where L is the dynamic range of the feature map and $K_1 = 0.01$, $K_2 = 0.03$. μ represents the mean of the patch. σ denotes the variance. Subscripts S and T are for student and teacher, respectively, and σ_{ST} is the covariance.

3.7 QUANTIZATION

Quantization is a technique used in machine learning model compression to reduce the memory footprint and computational requirements of deep neural networks. It involves representing the weights and activations of the network with fewer bits than their original floating-point precision. By quantizing these parameters to lower bit-widths (e.g., from 32-bit floating point to 8-bit integers), the model size decreases significantly, leading to reduced memory usage and faster inference times.

Despite the reduction in precision, quantization aims to minimize the impact on model performance by carefully choosing appropriate quantization strategies and optimizing the quantized model. With advancements in quantization-aware training and optimization techniques, quantized models can often achieve performance close to their full-precision

counterparts while offering significant benefits in terms of efficiency and deployment on resource-constrained devices.

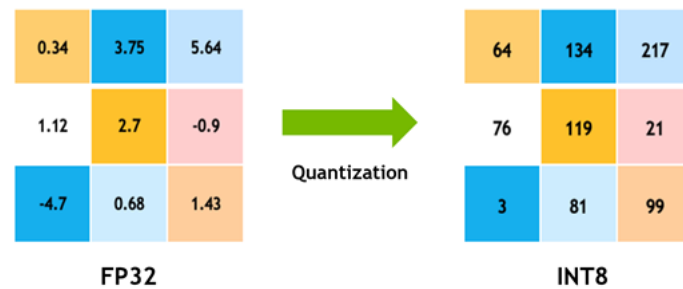


Fig 9

4. PROBLEM STATEMENT

Developing a robust and accurate object detection backbone to be deployed on the edge for autonomous vehicles and other advanced driver-assistance systems (ADAS) operating in adverse weather conditions, particularly haze and fog.

4.1 MOTIVATION

Safety: Reliable object detection is crucial for safe navigation in low-visibility road situations.

Technological Advancement: This research contributes to developing more adaptable autonomous driving technology requiring less computational power.

4.2 CHALLENGES

Limited Training Data: Collecting and annotating a large dataset of images with diverse weather conditions can be expensive and time-consuming. Such a dataset is also not available.

Domain Shift: Traditional object detection models trained on clear weather data struggle to generalize to hazy/foggy scenarios.

FPGA implementation: implementing models on FPGA has challenges as FPGA poses memory usage restrictions. (everything has to be determined at compile time. No dynamic structures. Limited memory)

Compression: Compressing a model presents a daunting challenge. Shrinking its size while preserving performance demands careful optimization. The right balance entails navigating

trade-offs between speed, memory usage, and complexity. Achieving this delicate balance requires innovative techniques to ensure the compressed model remains effective within the constraints of edge deployment.

5. OBJECTIVES

- 1. To identify the appropriate SpineNet backbone architecture for deployment on FPGA:** By testing the various architectures on a synthetic hazy dataset[3] to create a performance baseline for comparison.
- 2. To export the SpineNet architecture to RTL using Vivado/Vitis HLS and optimize it for enhanced latency/throughput:** By implementing the SpineNet backbone on Vivado/Vitis and using various directives to optimize the code.
- 3. To experiment for possible compression of selected SpineNet architecture** By using quantization, network pruning, knowledge distillation to reduce model size, etc.[5].

6. METHODOLOGY

Objective 1:

The primary objective of this phase of the project was to conduct a comprehensive evaluation of models within the SpineNet family—a class of neural network architectures renowned for their efficacy in object detection.

For the evaluation, Mean Average Precision(MaP) was chosen as the metric. The code implementation for MaP calculation was obtained from GitHub([link](#))[8]. This code implementation required the ground truth and detection results to be in a specific format (<class_name> <left> <top> <right> <bottom>).

Firstly, a parser was written to generate the ground truth files from the Cityscapes annotation format. After having the ground truth, it was time to generate the detection results.

The official implementation of SpineNet([link](#)) had the option to run inference. However, it ran inference only one image. Hence, the repository was cloned and modified to run inference on a whole directory of images, using SpineNet as the backbone instead of ResNet (default). Additional modifications were made to store the results in the format defined earlier for MaP calculation.

The results of these efforts are discussed in detail in section 7.1 of the report.

Objective 2

Objective 2 aimed to migrate the Spinenet architecture onto High-Level Synthesis (HLS), a task that unfolded a series of challenges and complexities.

The SpineNet 49S checkpoint file was downloaded from the official repository ([link](#)). The tensorflow checkpoint file contains only the model weights and not the architecture. This presented the first challenge: assign the 1040 weight matrices in the checkpoint file to the appropriate layers.

To do so, the architecture had to be understood in depth, down to the minute details of each sub-layer of the network. This necessitated a meticulous examination, consuming a substantial amount of time due to the complex nature of the network. Upon comprehending the network and the organization of the checkpoint file, another parser was written to extract the required weight.

Initially, the goal of this objective was to write the HLS code from scratch, which follows the C++ syntax. For this, a Python script was written that utilized the parser to generate C++ header files that stored each of the 1040 weight matrices as a C++ array with appropriate variable names.

After organizing the weights, the next step was to create the skeleton architecture of SpineNet 49S in HLS and load the stored weights from the header files. To approach this step, modular functions were created separately for each type of block(residual and bottleneck) and all the constituent functions of a convolutional network (activation, convolution, pooling, padding, batch normalization, concatenation, addition, upsampling).

At this point, a better alternative, hls4ml, was discovered. hls4ml is a Python package used to generate HLS code from the model file. This approach created the need to design and store a Keras model in .h5 format. The SpineNet was a complex network to design on Keras due to its key features, such as cross-scale connections and resampling operations. This feat was completed by modularizing the architecture through separate functions (as done for C++) and using the checkpoint parser to load the weights into the architecture. The design of the complete final model can be seen using the link in section 7.3.

Using the .h5 model, HLS code was generated, and C synthesis was attempted. Due to the large size of SpineNet 49S, the synthesis failed. To see the breaking point, the model was made smaller by removing layers one by one till the synthesis ran. This process reached five blocks and still failed. Then, at five blocks, all the parallelization directives of HLS were removed (Loop unrolling, array partitioning, Pipelining), and the synthesis ran successfully. The utilization report of the synthesis is provided in section 7.2. This clarified the need for compression for FPGA deployment.

Objective 3:

The objective focuses on the compression of the original SpineNet49S. Various strategies have been considered for this, such as quantization, knowledge distillation, pruning, etc. The work began with some analysis for quantization to determine the degree of precision required to represent the model fully. The plot for the same can be found in section 7.5.

The most promising avenue for this was structural knowledge distillation[6]. This method used the l_{ssim} loss metric instead of the more common l_2 norm. A distiller was created that used SpineNet49S as the teacher and a model as the student.

SpineNet49S became the inspiration for creating the student. The blocks of SpineNet were grouped and treated as one block. The input and output edges of the group were decided based on the input-output of the individual blocks. To determine the scale of the grouped block, the average was taken of all the constituent blocks while keeping in mind the order of scale variation of the SpineNet49S for exceptions.

In the resulting student, the number of blocks at each level was halved. The number of residual blocks was halved as well. The student ended up having eight blocks in total, including the stem blocks. For the exact architecture, please refer to section 7.4.

The foggy cityscape data was used to train the student with the help of the teacher. Due to resource exhaustion, training was not possible on the train set. Several attempts were made to use the train set. Data loaders with different batch sizes were experimented with, but RAM usage spiked, and the kernel crashed. This work was shifted to Google Collab Pro. However, the GPU and RAM exhaustion crashed the kernel there as well. To force the work ahead, the distillery was fitted on the partial train set of the dataset, which contained 7000 images on ten epochs.

The immediate next step is to evaluate this student for performance and make the necessary iterations in the model or the training parameters.

7. OBSERVATIONS AND FINDINGS

7.1. FOR OBJECTIVE 1: TO IDENTIFY THE APPROPRIATE SPINENET BACKBONE ARCHITECTURE FOR DEPLOYMENT ON FPGA.

For the purpose of on-road object detection in hazy conditions, MaP evaluation was carried out using the Cityscapes Dataset[2]. Furthermore, the foggy variation of the dataset (leftImg8bit_trainval_foggyDBF)[3] was tested. To obtain this data, synthetic haze/fog of 3 different intensities has been added to the original data.

Only a subset of all classes, Valid_Set = {car, bus, motorcycle, person}, was considered for metric evaluation due to the use case.

Experimentation utilized the following three architectures of the SpinNet Family[1]:

1. SpineNet-49
2. SpineNet-49S
- 3 SpineNetMB-49 (Mobile RetinaNet)

To create a baseline, The aforementioned variations of SpineNet were used to evaluate the original Cityscapes validation set(500 images without haze). Table 2 presents the outcomes.

After obtaining the baseline, the validation set of the hazy dataset was used to evaluate the MaP for all the classes(Table 4) and the classes in Valid_Set(Table 3)

MODEL	MaP(%)
SpineNet 49	44.23
SpineNet 49S	40.65

Table 2: MaP for Valid_Set on original data

MODEL	MaP(%)
SpineNet 49	36.91
SpineNet 49S	34.48
SpineNet 49MB	26.3

Table 3: MaP for Valid_Set on foggy data

MODEL	MaP(%)
SpineNet 49	31.53
SpineNet 49S	29.7
SpineNet 49MB	20.49

Table 4: MaP for all classes on foggy data

7.2. C SYNTHESIS

After designing SpineNet49S on Keras using functional API and saving it in .h5 format, the HLS code was generated using hs4ml. This was followed by attempts to run the C synthesis of the generated HLS code. Due to the enormous size of the SpineNet49S network, the synthesis did not run.

Hence, the model size was reduced block by block till the point it became viable to run the C synthesis. After a few iterations of reduction, five blocks of the model were reached. While reducing the number of blocks, the practicality of the output was not considered. Instead, only the resource utilization for different sizes of networks was observed.

Even at five blocks, the synthesis failed. At this point, the parallelization directives were removed to reduce the resource usage at one clock cycle. All of the loop unrolling, array partitioning, and pipelining pragmas were removed(commented). After this, the C synthesis is finally executed. The resource utilization at this point was still significantly high, as can be seen in Table 5.

NAME	BRAM_18K	DSP48E	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	-	0	86	-
FIFO	25149	-	360157	1325505	-
Instance	9	1	12068	19150	0
Memory	-	-	-	-	-
Multiplexer	-	-	-	1340	-
Register	-	-	206	-	-
Total	25158	1	372431	1346081	0
Available	624	1728	460800	230400	96
Utilization(%)	4031	0	80	584	0

Table 5. Utilization Estimates

7.3. SPINENET49S IMPLEMENTATION

The following is the partial diagram of the initial few layers of the SpineNet49S architecture on Keras. The complete network consists of a large number of layers. Thus, it has not been included in this report. The QR provided below can be used to access the complete network diagram.

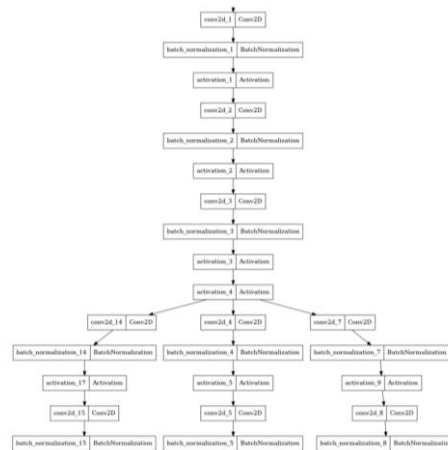


Fig 10. Initial layers of SpineNet49S on Keras

(Large diagram) [Link to the designed keras model diagram](#)



Fig 11. QR for Complete Network

7.4. STUDENT

The student model was made in an attempt to perform distillation of the teacher, SpineNet49S. It retains the scale permutations and cross-scale connections of the teacher network and is trained using the Foggy Cityscapes train set, which contains 8925 images. The training was executed for 10 epochs using a data loader with batch size 2.

The color coding of the architecture diagram is the same as the original SpineNet49S. The blocks highlighted in red represent the output blocks that are fed into the FPN (or RetinaNet). Diamond blocks represent the residual block, and rectangle blocks are the bottleneck blocks.

The first two blocks are the same as that of SpineNet49S as they are the stem blocks from the original ResNet 50 architecture.

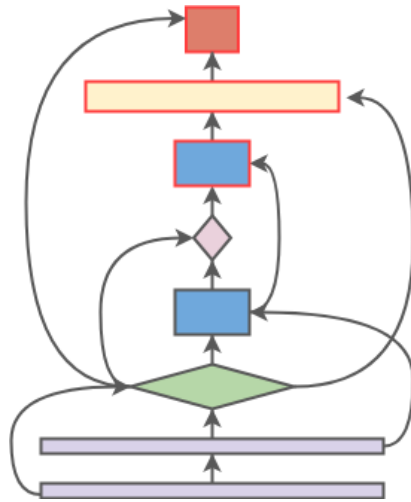


Fig 12 Student Model

7.5. QUANTIZATION

The following is the scatter plot for the weight value frequency, which visualizes the sparsity and range of weights.

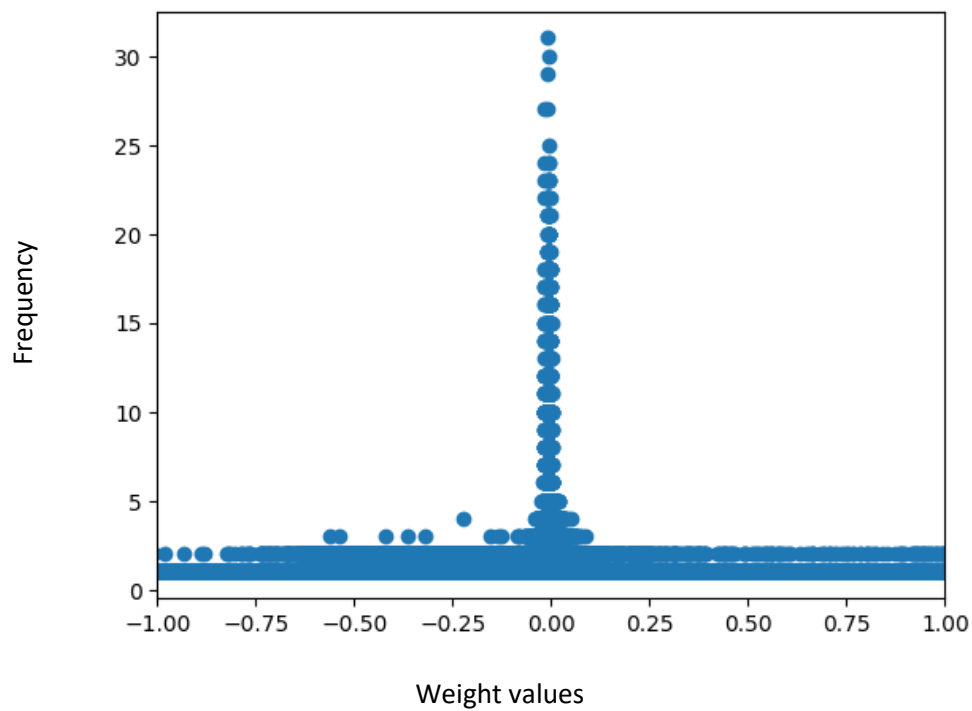


Fig 13. Weight frequency plot

8. PROGRESS

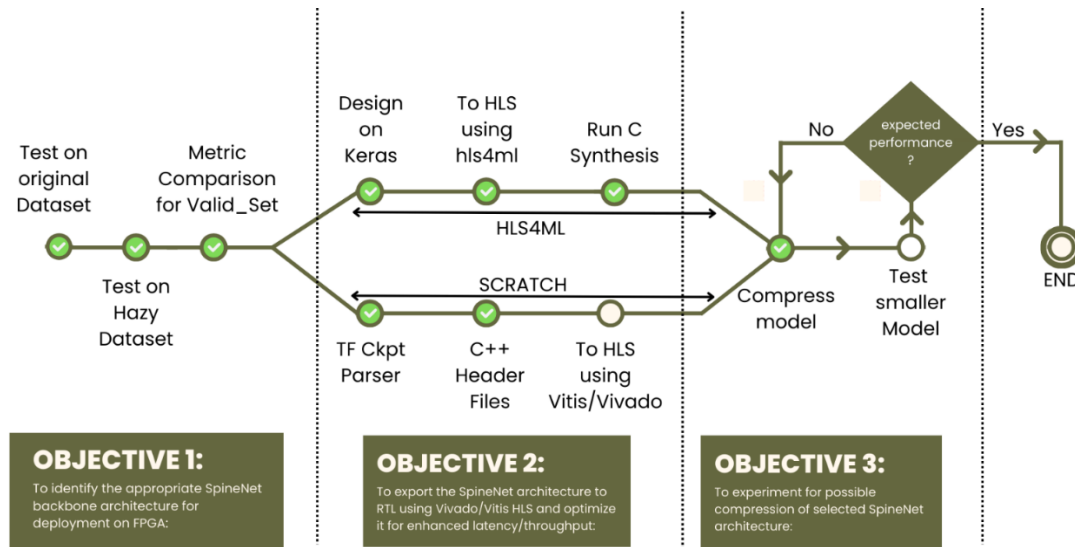


Fig 14. Progress Graph

Objective 1: COMPLETED

By

1. **(Completed)** Test on Original dataset
2. **(Completed)** Test on Hazy Dataset
3. **(Completed)** Comparing the metrics just for the Valid_Set of items

Outcome of Objective 1 => Move forward with SpineNet49S

Objective 2: COMPLETED

Initially, the work progressed with approach A. However, Approach A was halted as a better alternative was discovered.

The alternative, hls4ml, is a Python library that generates HLS code and runs C synthesis. It also provides functionalities for pruning and quantization. Due to ease of use and availability of required functionalities, the hls4ml approach was chosen as the preferred approach.

The HLS code was generated using approach B, and attempts to run the C synthesis were made.

APPROACH A: FROM SCRATCH USING VITIS HLS

By:

1. **(Completed)** Creating a parser for the TensorFlow checkpoint file.

2. **(Completed)** Using the parser to store the pre-trained weights as C++ header files.
3. Implementing the SpineNet49S architecture using the header files and Vitis Vision Library / HLS Video Library.

APPROACH B: HLS4ML

By:

4. **(Completed)** Designing the SpineNet49S architecture on Keras and exporting the h5 file.
5. **(Completed)** Using the hls4ml Package and the saved h5 file to generate the HLS code.
6. **(Completed)** Run C Synthesis of the generated code.

Outcome Objective 2: HLS4ML approach.

Objective 3: one iteration complete

This objective requires an iterative approach. This approach comprises designing smaller models and testing them to evaluate their performance. These two steps will be repeated till we reach our desired output. Knowledge distillation was the first method that was implemented.

By

1. **(Completed)** Designing a student model
2. **(Completed)** Creating a Distiller with l_{ssim} loss function and SpineNet 49S as teacher.
3. **(Completed)** Executing the distiller to train the student on the Foggy Cityscape dataset.
4. **(Completed)** Evaluating the student's performance.
5. Repeating steps 1 to 5 if required.
6. **(Completed)** Re-implementing on HLS using hls4ml.

9. LIMITATIONS

Nighttime Scenarios:

The original model demonstrates robust performance on both clear and hazy daytime data, and the student model has been distilled using hazy data. None of the data consists of night-time scenes. Consequently, the model may exhibit suboptimal performance in nighttime scenarios due to the absence of relevant training data.

FPGA Integration:

The network architecture comprises three main components: the backbone, neck, and head. This research focuses only on migrating the backbone network. This partial migration introduces a performance overhead as the neck and head components continue to run on a general-purpose processor, potentially limiting the overall efficiency of the system.

FPGA Memory Constraints:

FPGAs are characterized by limited memory capacity, posing significant challenges in migrating large neural networks onto them. This constraint hinders the ability to achieve high levels of precision and can impact the overall performance of the network when deployed on an FPGA.

10. CONCLUSION

To sum up, this effort has significantly advanced the field of study and laid a solid foundation for further research. The main accomplishment is the SpineNet implementation on Keras, which is a crucial development. This solution streamlines future research and development efforts by making it easier to experiment with the model and removing the need to wade through hundreds of lines of intricate code.

In addition, the project has produced a number of crucial tools and scripts that will be very helpful for future research. These consist of header file generation scripts, checkpoint parsers, and an annotation format conversion script. Each of these technologies fills a particular need in the research workflow, enabling more accurate and efficient model administration and data handling.

This project's introduction of model compression using distillation is another noteworthy accomplishment. A significant step towards building more compact and effective models has been taken with the development and application of the first student model. Future research aimed at creating tiny models that retain excellent performance levels is made possible by this foundational work.

All things considered, the progress made on this project not only improves the state of the art but also establishes a thorough foundation for further studies. The instruments and approaches created here will play a major role in the advancement and continuation of this field's study, guaranteeing more seamless and effective development in related projects in the future.

11. FUTURE WORK

11.1 MIGRATING OTHER PARTS OF THE NETWORK ON FPGA

The complete pipeline consists of the backbone, neck, and head. Current work focuses on deploying the backbone of FPGA while keeping other parts outside the board.

Immediate future work may involve extending FPGA utilization beyond the current scope. Migrating other parts of the object detection pipeline onto FPGA optimizes computational efficiency and accelerates overall processing speed. This expansion promises to enhance the system's performance.

11.2 OTHER SMALLER MODELS FOR KNOWLEDGE DISTILLATION

Similar to Experiment 8.6, we can derive more architectures from the SpineNet family or even conduct a literature survey for other object detection backbones.

The primary focus lies in making smaller variants while ensuring the retention of core properties. Another possibility is scouring existing literature for other models that perform well in hazy scenarios and using them for smaller models, ensuring that the resultant models maintain essential design elements despite reduced size.

Once formulated, knowledge distillation can be employed during training to transfer insights from the original bigger(teacher) network to the smaller(student). This approach enhances the efficiency of the smaller models while preserving their performance characteristics.

Subsequently, a thorough comparative analysis evaluates the performance of these compact models against established benchmarks. This examination provides essential feedback to refine the approach and further optimize neural network architectures for future applications.

11.3 NAS FOR SMALLER MODELS

Inspired by the teacher, Spinenet, the integration of Neural Architecture Search (NAS) methods stands out as a promising avenue. By employing NAS techniques, the development of smaller, more efficient models tailored to specific tasks can be streamlined. This can not only enhance computational efficiency but also facilitate the deployment of AI solutions across a spectrum of devices with varying computational resources.

12. REFERENCES

- [1] Du, X., Lin, T. Y., Jin, P., Ghiasi, G., Tan, M., Cui, Y., ... & Song, X. (2020). Spinenet: Learning scale-permuted backbone for recognition and localization. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (pp. 11592-11601).
- [2] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 770-778).
- [3] Sakaridis, C., Dai, D., Hecker, S., & Van Gool, L. (2018). Model adaptation with synthetic and real data for semantic dense foggy scene understanding. In Proceedings of the european conference on computer vision (ECCV) (pp. 687-704).
- [4] Cordts, M., Omran, M., Ramos, S., Rehfeld, T., Enzweiler, M., Benenson, R., ... & Schiele, B. (2016). The cityscapes dataset for semantic urban scene understanding. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 3213-3223).
- [5] Lyu, Z., Yu, T., Pan, F., Zhang, Y., Luo, J., Zhang, D., ... & Li, G. (2023). A survey of model compression strategies for object detection. *Multimedia Tools and Applications*, 1-72.
- [6] De Rijk, P., Schneider, L., Cordts, M., & Gavrila, D. (2022). Structural knowledge distillation for object detection. *Advances in Neural Information Processing Systems*, 35, 3858-3870.
- [7] X. Dai, Z. Jiang, Z. Wu, Y. Bao, Z. Wang, S. Liu, and E. Zhou. General Instance Distillation for Object Detection. CVPR, 2021.
- [8] Cartucho, J., Ventura, R., & Veloso, M. (2018). Robust object recognition through symbiotic deep learning in mobile robots. In 2018 IEEE/RSJ international conference on intelligent robots and systems (IROS) (pp. 2336-2341). IEEE.

Annexure A. Evaluation Form for Peer Review

Name of the student: (to be reviewed)	ADITYA ROY CHOWDHURY	Roll no. of the student:	102017144
<i>This form has to be submitted by the student whose roll no. will be mentioned in the box above. Handover this to the panel at the time of final presentation.</i>			
Title of the project:	FPGA BASED REAL-TIME DETECTION PIPELINE FOR CHALLENGING ENVIRONMENTS		
Name of the company:	INTERNATIONAL CENTRE FOR THEORETICAL PHYSICS		
Project report (Tick the appropriate)	Excellent 	Good	Average
Project poster (Tick the appropriate)	Excellent 	Good	Average
Project video (Tick the appropriate)	Excellent 	Good	Average
Rate the work done	0 – 10 points	(Provide rating here) →	10
Give marks to the student on the basis of the overall performance	0 -5 marks	(Provide marks here) →	5
Abstract of the project (max. 100 words): The report explores the importance of Object Detection in Artificial Intelligence, particularly for advanced driver-assistance systems (ADAS). It focuses on enhancing ADAS by deploying the SpineNet 49S architecture on a Field Programmable Gate Array (FPGA) board using Vivado HLS. However, due to the model's large size and FPGA resource limitations, the study also investigated compression techniques for the SpineNet backbone. The research resulted in a custom SpineNet implementation for easier experimentation and the development of a smaller student model inspired by SpineNet 49S.			
Mention three strengths of the work done: • Such a backbone on an FPGA will be a novel work. • Provides ground on future work for compression. • Gives the first generation of student model.			
Provide some useful recommendations (It may be some improvements, some suggestions to further raise the quality of the project):			

- RetinaNet could be moved on the edge board as well.
- More ways of compression can be explored.

Name of the evaluator student:	MUDRIKA JAIN	Roll no. of the evaluator student:	102017143
Signature of the Evaluator student:			